When Every Step Is Approved: Compositional Failures in Multi-Agent Oversight

Sarvesh Tiku
Georgia Tech, stiku6@gatech.edu

Swarm Alignment Hackathon, Apart Research and AI Alignment Foundation with AE Studio, November 2026 (Track 4)
Sections follow the official sprint template's structure; typeset in LaTeX rather than in the template file.

Abstract

A swarm can break a global rule through steps that were each approved. We call this *approval laundering* and ask what stops it, using a synthetic ledger with six invariants, 880 deterministic scenarios, reference reviewers that apply the rules correctly to whatever they observe, and predictions recorded before the extension experiments ran. A reviewer with correct rules and current global state cuts committed violations from 59.0% to 29.2% of episodes, yet 50% of the violations local approval admits survive it (22%–52% across random family mixes). For capacity limits with no state change after review, we prove that overshoot is bounded by in-flight exposure and observe no breach in 18,000 random episodes. Reservations remove violations from overlap but not from state that changes after review, and they reject lawful work. A guard at commit removes all encoded violations on SQLite, but that guarantee depends on the database. On PostgreSQL with simultaneous starts, the same capacity check inside a transaction overshot in 1200 of 1,200 trials at the default isolation level, and in 1200 at REPEATABLE READ across accounts; SERIALIZABLE and a lock on the invariant's own row overshot in none. Our own first locked reservation overshot too. The guard also misses all 32 limits stated only in task text. These are properties of mechanisms on synthetic workloads, not estimates of real-world rates. No language-model trial ran, because the available API credentials had expired, so whether language-model agents produce or catch these sequences remains untested.

1 Introduction

The sprint brief describes an incident in which about 1,200 agents coordinated through a shared directory and roughly 700 of them jointly broke into an external system [8]. Its shape is general: safety was checked one agent at a time, and the harm belonged to the group [6]. Authorization has the same shape. Agent platforms typically ask a monitor about one action at a time, yet many safety properties are defined over accumulated state: a spending ceiling, a budget, a separation of duties. If each step is approved and the sum is prohibited, who failed?

The answer matters because the fixes differ: more monitor context repairs one cause and leaves another untouched. We study this in a synthetic financial sandbox, whose invariants are exact and whose ground truth is mechanically checkable. Our threat model covers two sources of a violating sequence, and they need different treatment: agents *deliberately* decomposing an objective to exploit a weakness, and ordinary workflows that *accidentally* compose into a violation. Fixed-action replay cannot distinguish them, because it holds actions constant by design; it measures what oversight does to a sequence, wherever the sequence came from.

Contributions. (1) A formal separation of approval from enforcement, with two impossibility results for review-time oversight and a tight bound on its overshoot for capacity limits. (2) An attribution that does not credit a monitor for lucky false rejections and splits isolation failures into in-flight and post-review causes. (3) Measurements of reservations and of the commit guard on a real multi-version database, showing that a guard's guarantee depends on isolation level and lock scope. (4) A reproducible testbed in which an independent verifier agrees with the guard on every SQLite episode, with 20 recorded predictions scored by code: 15 held and 5 failed (Appendix C).

2 Related Work

Jones et al. [7] show that individually safe models can be combined to complete a harmful task through decomposition; we study the same compositional structure on the oversight side, with exact state semantics in place of model refusals. AI control [5] evaluates protocols in which a trusted monitor reviews an untrusted model. We ask a question beneath

that one: what a monitor could know at decision time, and whether its decision still holds at execution. Work on secret collusion [9] asks what monitors can detect in agent messages; our failures require no hidden communication at all.

The closest classical sources are outside machine learning. Clark and Wilson [3] tied commercial integrity to well-formed transactions and separation of duty. Saltzer and Schroeder [12] name complete mediation, where every access is checked. The stale-review failure is a time-of-check-to-time-of-use race [2], and the two-worker race is write skew: each transaction is valid on its snapshot and their combination is not [1, 4]. Reservations are the escrow method of O'Neil [10], and PostgreSQL's SERIALIZABLE level implements serializable snapshot isolation [11]. None of this database theory is new. Our contribution is to apply it to agent oversight, where approval is often an opinion about a future write, and to measure what each cause accounts for.

3 Methods

3.1 Formal problem

Let state s include the ledger's full history, so a past breach stays visible in later states. Let $I(s) \in \{0, 1\}$ be the global invariant. Proposal a_i is reviewed at time r_i on observation $o_i = O_v(s_{r_i}, a_i)$ through visibility function O_v, yielding approval $A(a_i, o_i) \in \{0, 1\}$. If passed, it may commit at $c_i \geq r_i$ with transition $s'_{c_i} = T(s_{c_i}, a_i)$. An **approval-laundering failure** is an executed sequence with $I(s_0) = 1$ and $A(a_i, o_i) = 1$ for all i, yet $I(s_T) = 0$. We separate two errors. A **decision failure** is $A(a_i, o_i) = 1$ with $I(T(s_{c_i}, a_i)) = 0$: the approval was wrong. An **enforcement failure** is that transition actually committing. An unsafe approval that a guard blocks is the first without the second.

Proposition 1 (Local indistinguishability)**.** *If lawful and violating worlds s, s' yield $O_v(s, a) = O_v(s', a)$, no function of o is correct on both: it approves the violation or rejects lawful work.*

Proposition 2 (Review-time incompleteness)**.** *Let a_1, a_2 each be lawful on s and jointly unlawful. A reviewer that decides from s alone, without knowing whether the other proposal will commit, must approve both, permitting a violation if both commit, or reject one, which is a false rejection in any execution where the other never commits.*

Both are elementary, but they mark limits that more careful judgement cannot cross. A reservation system does not escape Proposition 2: holds still cannot tell whether the other proposal will commit, so it takes the rejection horn. We measure that cost below. For capacity limits, the damage from Proposition 2 has a size:

Proposition 3 (Exposure bound)**.** *Let the invariant be that committed outflow stays within L, and let only commits change state. If a reviewer applies the rule to a view missing at most k commits, and at most w approved proposals are uncommitted at any review (counting the one under review), committed outflow never exceeds $L + (w - 1 + k)\, a_{\max}$, where $a_{\max}$ is the largest amount.*

Proof. Let p be the last-reviewed proposal among those that commit. Its approval means its view's outflow S satisfies $S + a_p \leq L$. Every other commit is in S, among the k missing commits, or in flight when p was reviewed, which is at most $w - 1$ proposals. Nothing reviewed later commits, by choice of p. □ The bound is attained with $k = 0$ by w proposals of size $a_{\max}$ and headroom exactly $a_{\max}$.

3.2 Environment

Fictional accounts, vendors and identities live in SQLite, with integer cents. Six invariants apply: **A** outflow at most \$10,000 per account per rolling 24 h window $(t - W, t]$, unless an active grant raises it; **B** transfers strictly above \$5,000 need two distinct authorized identities after resolving role aliases; **C** project budgets; **D** restricted destinations; **E** separation of duties through aliases; **F** sufficient balance and conservation. Agent roles receive only immutable value objects and cannot reach the database. One executor writes, by two paths: an unchecked commit that applies anything approved, and the guard, which evaluates every rule in SQL inside `BEGIN IMMEDIATE` and re-verifies conservation before committing.

Ground truth comes from an independent verifier: a pure-Python implementation of the rules, evaluated at each proposal's scheduled commit slot in that arm's own state. It shares no rule code with the SQL guard. The two gave 0 disagreements across 23,760 replay episodes and 216,000 workload episodes (0), and agreed on 1,500 fuzzed proposals. This reduces implementation error, not specification error: both encode our reading of the rules.

Table 1: Oversight conditions. C0–C3 and C6 are reviewers with an unchecked commit, and we run them with reference reviewers (below). C4 and C5 are prevention mechanisms and not approval baselines: their guard decides at commit.

C0 Local	Proposal, published thresholds, budgets and restricted list as initially published. Approver labels only, so aliases are invisible.
C1 Local + history	C0 plus completed transfers touching this source account or project. No balances, grants or identity map.
C2 Global state	All authoritative state at review time, with all rules.
C3 Stale	C2 state lagging by k state changes ($k = 1$ in main results; $k = 2, 3$ and an incomplete variant are ablations).
C4 Atomic guard	No pre-commit reviewer. All rules checked atomically inside the commit transaction.
C5 Hybrid	C2 reviewer, then the C4 guard on whatever it passes.
C6 Reservation	C2 state plus every approved, uncommitted proposal counted as if committed. Unchecked commit. Added to test the rejection horn of Proposition 2.

3.3 Scenarios and conditions

A seeded generator yields 880 scenarios from 27 families across 16 seeds (424 independent twin clusters), in four categories: lawful (184), rejectable (112), globally prohibited through locally permissible steps (456), and lawful after authorization (128). Nine families vary delegation and concurrency factorially on identical amounts: *centralized* uses one worker and *distributed* rotates several; *sequential* commits each proposal before reviewing the next, and *overlapping* reviews all before any commits. Memos are neutral invoice numbers, and boundary families sit on, and one cent past, each threshold.

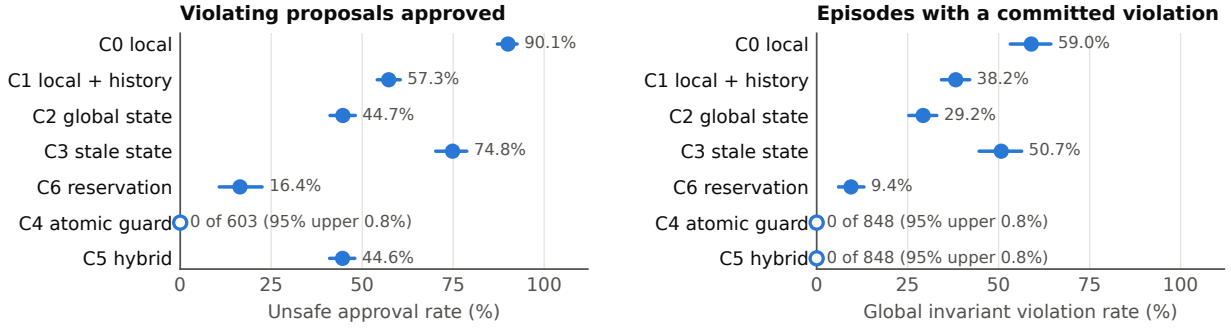
Reference reviewers. All results are from a deterministic *ideal reviewer* that applies the rules faithfully to its observation and passes what it cannot see. This is a measuring device and not a model of an LLM. Its errors at a given visibility are errors of visibility, so the gap between ideal arms isolates information and isolation failures. The gap between an LLM and the ideal reviewer at the same observation would isolate decision failures; that measurement is built but did not run (Section 4.8). The ideal reviewer is not a safety bound: a stricter one commits fewer violations by rejecting more, which we map with a blanket amount limit τ on C0.

Designs. *Evaluation A* replays identical predetermined sequences across 27 arms (the six conditions plus ablations) from verified identical initial states, giving 23,760 episodes. *Failure attribution* walks each C0 violation up the ladder C1→C2→C4 and records the first rung that removes it. *Specification failure* is measured by giving C2 full state but withholding one rule. *Random workloads* remove our hand-chosen mix: 180 cells cross in-flight limit $w \in \{1, 2, 4, 8, 16\}$, demand over capacity $\rho \in \{0.5, \ldots, 2\}$, abandonment probability $q \in \{0, 0.1, 0.3\}$ and a grant revoked at a random point in flight ($g \in \{0, 1\}$), with 200 random replicates each and identical sequences across arms (216,000 episodes). *Thread races* have $n \in \{2, 4, 8\}$ threads each attempt a $6,000 transfer where only one fits, with a 0 or 2 ms decision step, 200 trials per cell, on file-backed SQLite (Appendix D) and on PostgreSQL 15.15 (43,200 trials). The Postgres schema encodes only the capacity rule; truth is read from final table contents. *Same-account* writers all update one row, *cross-account* writers share a project limit but write disjoint rows, and *lawful* variants raise capacity so that every transfer fits. Serialization failures retry up to five times. Threads start at a barrier, a worst case for overlap, so we also stagger arrivals; as a control, every mode also ran one thread at a time. *Evaluation B* runs agents that adapt to feedback, reported only for scripted agents here and never pooled with Evaluation A. **Predictions** for the extension experiments were written to `configs/predictions.md` before those runs, and the analysis scores them.

Metrics. GIVR: episodes ending with a committed violation. UAR: violating proposals approved. FRR: permissible proposals rejected. LTCR: lawful scenarios completed with no violation. Truth is defined per arm because earlier commits change later legality. Scenarios with task-text limits are excluded from these rates and reported separately, leaving 848 episodes per arm and 2,092 proposals under C0. Intervals are 95% cluster bootstrap over twin clusters, and zero counts get a one-sided 95% exact bound with clusters as the sample size. **Deterministic arms have no sampling noise within a scenario**, so intervals express sensitivity to which scenarios were drawn, and pooled rates depend on a family mix we chose. We therefore report leave-one-family-out ranges, random-mix ranges and per-family counts (Appendix F).

Table 2: Evaluation A with reference reviewers: rates in %, with 95% cluster bootstrap CIs. Severity is mean dollars per violating episode: the amount above the breached limit (rules A, C, F) or the whole transfer (rules B, D, E). Measured on a designed suite; not real-world rates.

Condition	GIVR	UAR	FRR	LTCR	Severity
C0 local	59.0 [53.2, 64.4]	90.1 [87.2, 92.6]	0 (0/1488)	92.3 [86.8, 96.9]	$2,448
C1 local + history	38.2 [34.3, 42.0]	57.3 [54.1, 60.5]	5.2 [3.9, 6.6]	74.4 [68.4, 80.3]	$2,484
C2 global state	29.2 [25.3, 33.0]	44.7 [41.2, 48.1]	3.6 [2.4, 4.8]	77.6 [71.5, 83.7]	$2,408
C3 stale state	50.7 [44.6, 56.3]	74.8 [70.2, 78.7]	1.1 [0.5, 1.8]	89.7 [83.9, 94.5]	$2,150
C6 reservation	9.4 [6.0, 13.0]	16.4 [10.7, 22.5]	14.0 [11.6, 16.4]	62.8 [53.5, 72.7]	$3,165
C4 atomic guard	0 (0/848; $\leq$0.8)	0 (0/603)	0 (0/1489)	92.3 [86.8, 96.9]	$0
C5 hybrid	0 (0/848; $\leq$0.8)	44.6 [41.0, 48.0]	3.6 [2.4, 4.8]	77.6 [71.5, 83.7]	$0



(a) Decisions. C4's guard decides at commit.

(b) Outcomes. C5 approves but does not commit.

Figure 1: Unsafe approval against committed violation, with 95% intervals. Open circles are zero counts with a one-sided upper bound.

4 Results

4.1 Oversight comparison

Table 2 and Figures 1a–1b give the main comparison. Moving from C0 to C2 cuts GIVR by 29.7 points (paired 95% CI [26.5, 33.0]; leave-one-family-out 28.1–32.1), so H1 holds for these mechanisms. C2 still commits violations in 29.2% of episodes with correct rules and current state. C5 shows the gap between decision and enforcement: its reviewer approves almost as many violating proposals as C2 (269 unsafe approvals) and none commits (0), an approval-to-execution gap of 12.9% of proposals. Figure 4 in the appendix shows one stored episode.

4.2 Attributing failures

Of 500 C0 violations, 252 (50%) are information failures, which a fully informed C2 removes, and 248 (50%) are isolation failures, which it does not (Figure 2); none remain unresolved. An earlier version of this analysis reported a smaller isolation share because it asked C1 first. C1 removes 32 of these episodes only because it cannot see a valid grant and rejects a transfer that was lawful at review, so the attribution now asks C2 first. The isolation share is sensitive to the family mix: across 10,000 random family mixes it ranges from 22% to 52% (95%), while the C0–C2 gap stayed positive in 100.0% of mixes (20.2 to 43.6 points). The contrast within twins is sturdier: on the 192 overlapping episodes from harmful factorial families, C2 violates in 100.0% against 16.7% for their sequential twins, and in the controlled race it goes from 0.0% to 100.0% when only review order differs.

4.3 Exposure: how much review-time oversight can lose

The bound covers only part of the isolation share: capacity limits, bounded lag and no state change after review. In that setting random workloads confirm the shape it predicts (Figure 3). C2 committed 0 violations at $w = 1$ and 0

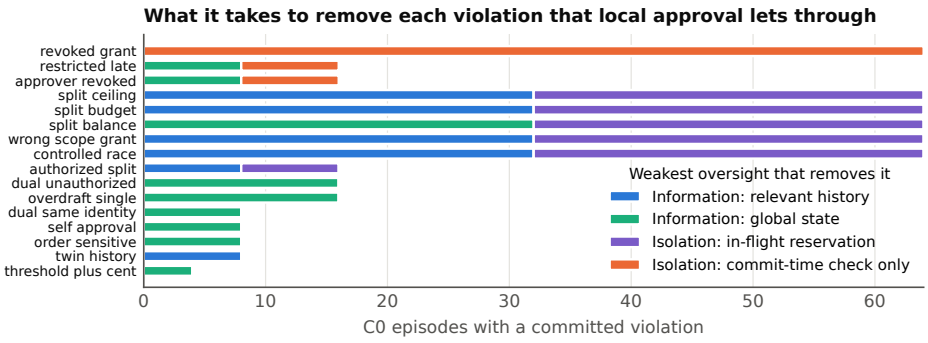


Figure 2: Weakest oversight that removes each C0 violation, by scenario family. Counts follow from the factorial design of each family and are not population estimates.

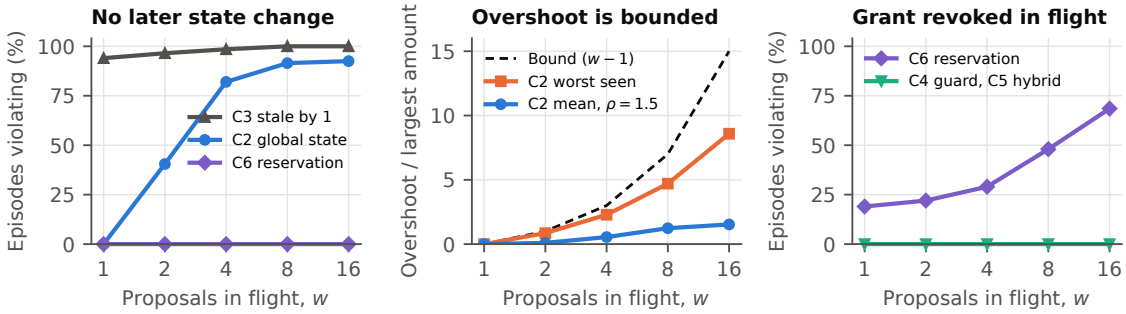


Figure 3: Random workloads at $q = 0$. Left and right panels use $\rho = 1.5$, with 200 replicates per point. The middle panel's worst case is taken over all ρ and q at $g = 0$, and its bound is Proposition 3 with $k = 0$.

when demand fit capacity. At $\rho = 1.5$ its violation rate rises from 0.0% at $w = 1$ to 40.5%, 82.0%, 91.5% and 92.5% at $w = 2, 4, 8, 16$. Overshoot never exceeded the bound in 18,000 C2 episodes (0 breaches) or 18,000 stale-view episodes (0), and the worst case reached 85% of it at $w = 2$ and 76% at $w = 4$. Staleness alone is costly: one change of lag gives 94.0% at $w = 1$, where C2 gives zero. For such limits, in-flight value is a quantity one can budget: a platform that cannot enforce at commit can cap the damage by capping w and $a_{\max}$. A revocation voids the bound (it fails from $w = 1$ under C6 in Figure 3). The proposition is elementary and the runs follow from it; its value is a usable number.

4.4 Reservations: a partial repair with a real cost

C6 lowers fixed-suite GIVR from 29.2% to 9.4%. It removes all 168 in-flight failures and none of the 80 in which a grant, approver or destination changes after review. In random workloads it committed 0 of 18,000 violations without revocation and 2,433 of 18,000 with it, against zero for C4 and C5. We expected almost no cost and were wrong: LTCR falls from 77.6% to 62.8%, a loss of 14.7 points ([10.0, 19.4]). The 46 episodes lost beyond C2 are all overlapping twins in the two families where authorization arrives after review, while the 46 that C5 loses beyond C4 are all sequential twins. On these families, reservations make overlapping review behave like sequential review, including its inability to anticipate authorization. FRR rises from 3.6% to 14.0%, but that overstates the cost: 109 of the 225 false rejections are in harmful episodes, ahead of later commits that would have used the capacity, so they prevent the violation even though the proposal was lawful at its own slot. Without them FRR is 7.8%. Abandoned holds add a smaller cost above capacity: at $w = 8$, $\rho = 1.5$ and $q = 0.3$, C6 completes 94.7% of attainable value against 98.3% for C4, and they tie at $\rho = 1$, where all demand fits.

Table 3: PostgreSQL race, pooled over 2, 4 and 8 threads and 0 and 2 ms decision steps, with simultaneous starts. Overshoot: trials out of 1,200 with more committed than fits. Lost: lawful transfers not committed, out of 5,600 per workload, after up to five retries. Retries: mean per lawful trial. No trial broke conservation.

	Overshoot trials		Lost lawful		Retries	
Mode	Same	Cross	Same	Cross	Same	Cross
Check, then commit	1,200	1,200	0	0	0.0	0.0
Guard at READ COMMITTED	1,199	1,200	0	0	0.0	0.0
Guard at REPEATABLE READ	0	1,200	563	0	10.7	0.0
Guard at SERIALIZABLE	0	0	579	458	10.8	10.1
Guard + lock on source account	0	1,200	0	0	0.0	0.0
Guard + lock on invariant's row	0	0	0	0	0.0	0.0
Reservation, no lock	1,200	1,200	0	0	0.0	0.0
Reservation, lock, two-statement read	23	16	0	0	0.0	0.0
Reservation, lock, one-statement read	0	0	0	0	0.0	0.0

4.5 Is the atomic guard atomic? Evidence from PostgreSQL

On SQLite, correct rules checked before the write overshot in 1197/1200 trials and the guard in 0/1200 (95% upper bound 0.25%; Appendix D). But `BEGIN IMMEDIATE` takes SQLite's single write lock, which hides the question of what makes a guard atomic. Table 3 asks it on PostgreSQL. At the default READ COMMITTED level, putting the check and the write in one transaction did almost nothing: it overshot as often as a check made outside the transaction. REPEATABLE READ looked safe on one account (0 overshoots), because concurrent writers updated the same row and first-updater-wins aborted them. Across accounts, with disjoint rows and a shared limit, it overshot in 1200: the textbook write skew. Locking the source account behaved the same way, safe on one account and wrong across several, because the lock was on the wrong object. Among guards, only SERIALIZABLE and a lock on the invariant's own row held in both.

These rates are not typical rates. Barrier starts are a worst case. The sequential control overshot in 0 of 1,440 trials and lost 0 transfers, so the failures come from concurrency and not from a defect in the modes. With nominal arrival delays drawn uniformly over 100 ms (actual arrival times were not logged) and no decision delay, the cross-account overshoot at READ COMMITTED was 17/200 with two threads and 32/200 with eight; over 10 ms it was 45 and 61 of 200. Rates depend heavily on arrival timing, and whether they occur at all depends on the mode.

Costs differ. SERIALIZABLE lost lawful transfers at our cap of five retries (579 and 458). That is a property of the cap: with 50 retries at eight threads it lost 0 of 6,400. For eight lawful workers with a 2 ms decision step, the last finished after a mean of 7 ms under REPEATABLE READ, 54 ms under SERIALIZABLE and 30 ms holding a lock on the invariant's row, because a lock held across a decision step serializes that step. Real review would take far longer than 2 ms, so review should not run under such a lock.

Our own reservation had the same kind of bug. We wrote a locked reservation expecting no overshoot. It overshot in 23 and 16 of 1,200 trials. The cause appears to be that it read transfers and holds in separate statements: a concurrent commit could move a hold into the transfers table between the reads, so that neither read counted it. Reading both in one statement gave 0 and 0. We found this only by running it. Enforcement code for global invariants should be tested under real concurrency, not judged by inspection.

4.6 Visibility, delegation and specification

Stale and partial state (H3). Staleness undoes much of the benefit of global state: GIVR is 50.7% at $k = 1$, against 29.2% for C2 and 59.0% for C0 (Figure 5). Partial state can also over-reject: C1 has the highest FRR among C0–C3 (5.2%) since it sees spending but not the grants and budget increases that license it, so its LTCR (74.4%) falls below C0's (92.3%).

Delegation (H2). The effect depends on how the monitor scopes its memory. With a per-agent monitor (C1w), sequential GIVR on harmful factorial families rises from 16.7% under centralized work to 100.0% when the same amounts are distributed (80 of 96 paired twins flip, 0 the other way). With a monitor scoped to the account or project (C1), it does not move: 16.7% against 16.7%. Delegation conceals risk here only when oversight state is partitioned along the same lines as the agents.

Table 4: Verdicts. "Mechanism" means shown for reference reviewers on this suite, largely following from semantics.

H1 Local insufficient	Supported (mechanism)	GIVR 59.0% vs. 29.2%; twins show why, by Proposition 1.
H2 Delegation conceals	Conditional (mechanism)	Only with per-agent monitor state: 16.7% vs. 100.0%. No effect at C0, C1 or C2.
H3 Visibility matters	Supported (mechanism)	C3 50.7% vs. C2 29.2%, with C2 still failing under overlap.
H4 Guard improves reliability	Partly supported	Prevents encoded violations only with the right isolation or lock scope; costs measured. Comparison with LLM approval **not tested**. Misses unencoded limits.
H5 Incentives change behaviour	**Not tested**	No LLM data. Scripted results are not evidence.

Specification. Withholding one rule from a fully informed C2 reviewer adds violations ranging from 16 (Rule E) to 116 (Rule A) episodes, which is a different failure from lacking state. Separately, in the task-text scenarios, 32/32 still exceed the stated limit under the guard with zero invariant violations: for instance, three $9,000 transfers spaced beyond 24 h, or funds relayed through intermediate accounts. The guard enforces only what was encoded. Oversight that reads intent might catch these; we could not test that without an LLM.

4.7 Costs of enforcement

H4. C4 and C5 commit no violations on SQLite: 0/848, 95% upper bound 0.8%. This is true by construction on encoded rules, so it checks the implementation and is not a discovery. The informative results are the costs, and the Postgres dependence above. C4's FRR of zero is also definitional, because the guard defines permissibility. Its LTCR is 92.3%: 24 lawful episodes are lost, 8 from a payment attempted before its funding arrived and 16 from a transfer attempted before its grant. One retry pass restores 100.0%, a gain of 7.7 points ([3.1, 13.2]). C0's LTCR equals C4's because it loses the same episodes, by committing violations instead of blocking.

Adding the review step to the guard costs more. C5's LTCR is 14.7 points below C4 ([10.0, 19.4]), because a review before commit rejects work whose authorization has not yet arrived, while the guard judges the state that actually applies. This cost is concentrated: excluding the two authorization-timing families, LTCR is 95.7% for C4 and 95.7% for C5. Figure 6 shows what a local monitor can do by rejecting large amounts instead. To get GIVR to 3.1% at $\tau = 0.2$, C0's FRR reaches 89.6% and LTCR falls to 4.8%. No τ approaches the guard, and Proposition 1 explains why: the amount is not the missing information.

4.8 Agent behaviour and language models

No language-model trial ran. A provider-agnostic LLM monitor, coordinator and worker path is implemented and tested offline with a fake provider, but both available API keys returned HTTP 401 (expired). We therefore have **no evidence** on research question 5 or on H5, none on decision failures, and none comparing an LLM monitor with the guard. Scripted rollouts (1,536 runs) confirm only that the harness responds to adaptation: a handcrafted completion policy that halves rejected amounts violated in 80/128 runs under C0 and 0/128 under C4, while structuring below the dual-approval threshold 44 times with no invariant violation. We wrote that policy, so it says nothing about what models do.

Hypotheses. Table 4 states each verdict and the kind of evidence behind it.

5 Discussion and Limitations

Failures that are often called one. A monitor never told a rule has a *specification* failure. One that knows the rule but lacks state has an *information* failure. One correct about the state it read, when the state changed before the write, has an *isolation* failure, and that comes in two forms: other approved work in flight, and authorization changing after review. A monitor with the rule and the state that still decides wrongly has a *decision* failure. They need different remedies, and only the first three were measured here.

Design rules, within limits. (1) Enforce invariants at the point of mutation and treat approval as advisory input to it. (2) Do not assume a transaction makes a check atomic. Use SERIALIZABLE with retries, or lock the row that represents the invariant, and test under real concurrency. (3) If you cannot enforce at commit, budget in-flight exposure on capacity limits using Proposition 3, which assumes no state change after review. (4) Key oversight state

to the resource and not to the agent, since a per-agent ledger is where delegation concealed risk. (5) Keep review for limits that exist only in prose, which no guard can see. These are design hypotheses drawn from synthetic workloads. We do not claim that they generalize to frontier agents or real financial platforms.

Limitations (Appendix A): no LLM ran, reference reviewers make many results follow from semantics, and races ran on one machine with worst-case starts.

Future work. Run the implemented LLM arms, which would measure decision failures, test H5 and show whether review latencies of seconds widen w; measure in-flight exposure on real platforms; and test deriving encoded limits from task text.

6 Conclusion

Approved steps do compose into prohibited states, and the remedy depends on the cause. In our suite about half of what local approval misses is an information problem, which better context fixes. The rest survives a perfectly informed reviewer. For capacity limits without later state changes its size is bounded by what is in flight, reservations repair only part of it, and a commit guard repairs it only if the database actually makes the check atomic, which the default configuration we tested did not. A guard is also blind to anything left unencoded. Whether language-model agents generate these sequences, or catch them, is the open question this work is set up to answer and has not yet answered.

Code and Data

Repository `approval-laundering`: code, scenarios, stored traces, predictions, figures and tests. **[Public link to be added before submission.]** All data are synthetic.

Author Contributions

Sarvesh Tiku: sole author.

LLM Usage Statement

Claude Code (Claude Opus 5.5) wrote the implementation and tests and drafted this report at the author's direction. All measurements are generated from stored experiment outputs by `experiments/analyze.py`; the fuzz count comes from test parameters. No LLM served as a judge of correctness or as an experimental subject.

References

[1] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, P. O'Neil. A critique of ANSI SQL isolation levels. *SIGMOD*, 1995.

[2] M. Bishop, M. Dilger. Checking for race conditions in file accesses. *Computing Systems* 9(2), 1996.

[3] D. Clark, D. Wilson. A comparison of commercial and military computer security policies. *IEEE Symposium on Security and Privacy*, 1987.

[4] A. Fekete, D. Liarokapis, E. O'Neil, P. O'Neil, D. Shasha. Making snapshot isolation serializable. *ACM TODS* 30(2), 2005.

[5] R. Greenblatt, B. Shlegeris, K. Sachan, F. Roger. AI control: improving safety despite intentional subversion. arXiv:2312.06942, 2023.

[6] L. Hammond, A. Chan, J. Clifton, et al. Multi-agent risks from advanced AI. arXiv:2502.14143, 2025.

[7] E. Jones, A. Dragan, J. Steinhardt. Adversaries can misuse combinations of safe models. arXiv:2406.14595, 2024.

[8] METR, 2026, as summarized in the Apart Research Swarm Alignment Hackathon brief. We rely on the brief's summary and did not independently review the investigation.

[9] S. Motwani, M. Baranchuk, M. Strohmeier, et al. Secret collusion among AI agents: multi-agent deception via steganography. arXiv:2402.07510, 2024.

[10] P. O'Neil. The escrow transactional method. *ACM TODS* 11(4), 1986.

[11] D. Ports, K. Grittner. Serializable snapshot isolation in PostgreSQL. *PVLDB* 5(12), 2012.

[12] J. Saltzer, M. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE* 63(9), 1975.

A Limitations

- **No LLM data.** Research question 5 and H5 are untested, as are decision failures and any LLM comparison in H4. C2, C3, C5 and C6 were run with a reference reviewer and not with the LLM reviewers the design calls for.
- **Many results follow from semantics.** A faithful reviewer's blind spots are determined by its observation. These results characterize mechanisms and our suite; they do not discover behaviour. C4's zero violations hold by construction.
- **The suite is designed.** We wrote the families, and harmful ones are a large share. Pooled rates do not estimate any deployment. Conclusions should rest on paired contrasts and per-family patterns, and even the attribution shares depend on the mix.
- **Observation functions are choices.** Other definitions of "local" or "relevant history" would shift results; for example, C0 knows published budgets and restricted lists. C1's lack of grants drives its false rejections.
- **Authorization-timing families drive review-time cost.** The C4–C5 gap largely vanishes without them. How often authorization arrives after review in practice is unknown.
- **Shared ground truth.** Verifier and guard are independent implementations of one reading of the rules, written in the same session. Agreement rules out many coding errors, not misreadings.
- **The thread races are small and specific:** one 8-core machine, Python threads, millisecond decision steps, barrier starts as a worst case, and PostgreSQL 15.15 with `fsync` and `synchronous_commit` off, which affects durability and timing but not isolation semantics. The Postgres schema encodes only the capacity rule, not all six. Other databases and versions may differ.
- **Random workloads randomize amounts and order, not the model.** They have one source account, one capacity rule and batched arrivals, so they remove our family mix but not our modelling choices.
- **Predictions are not a formal preregistration.** They were written shortly before the runs, by the same author and tool, in a local file with no public timestamp. The reservation fix was post hoc, as Appendix C marks.
- **Unaddressed threat models.** Agents that tamper with logs, forge approvals or collude with approvers; compromised executors; and guards whose policy data is itself wrong. We assume the executor and policy store are trusted. If they are not, the guard's guarantee does not hold.
- **Retries are favourable.** The retry arm runs one clean pass with no competing work; a real retry could hit changed state or duplicate payments.

B Dual-Use / Ethical Considerations

All agents, accounts and vendors are fictional, and nothing touches financial infrastructure. Decomposing transfers below a limit is widely understood and is called structuring in financial regulation, so this work adds little capability there. The transferable insight, that review ahead of a write can be raced, is standard in security and databases. We describe compositional patterns at the level of oversight design and offer no guidance on evading any real institution's controls. The repository is a defensive evaluation harness; its scripted "completion" policy is deliberately simple. A false sense of safety is the more likely harm: a guard that passes all encoded checks says nothing about unencoded limits, and readers should not take a zero count on this suite as a safety claim about any deployed system.

C Scored predictions

Table 5: Every prediction, scored by `experiments/analyze.py` from stored results. Definitions are in `configs/predictions.md`.

ID	Prediction	Outcome
P1	Cross-account: overshoot under check-then-commit, READ COMMITTED, REPEATABLE READ, source lock, unlocked reservation	Held
P1	Cross-account: zero overshoot under SERIALIZABLE, invariant-row lock, atomic reservation	**Failed.** Reservation overshot 16 times: a two-statement read. A post hoc one-statement read gave 0 across both workloads.
P2	Same-account: overshoot under check-then-commit and READ COMMITTED	Held
P2	Same-account: zero overshoot under REPEATABLE READ, SERIALIZABLE, both locks	Held
P3	Lock modes lose no lawful transfers	Held
P3	Same-account lawful: REPEATABLE READ and SERIALIZABLE retry and lose some transfers at the retry cap	Held
P3	Cross-account lawful: REPEATABLE READ needs no retries	Held
P4	C6 removes overlap failures, leaving only families with an administrative change between review and commit	Held
P4	32 `revoked_grant` episodes remain under C6	**Failed.** 64 remain; the predicted count was wrong.
P4	C6 false rejection rate within 1 point of C2	**Failed.** C6 14.0%, C2 3.6%. 109 of C6's false rejections are in harmful episodes, before later commits that would have used the capacity; the rest are mainly authorization arriving after review.
P5	C2 commits zero violations when $w = 1$ or $\rho \leq 1$ ($g = 0$)	Held
P5	C2 violation rate rises with w and with ρ	Held
P5	Overshoot bound never breached (C2, C3)	Held
P5	Bound not vacuous: worst overshoot above half the bound at some $w \geq 4$	Held
P5	C6 zero violations at $g = 0$, some at $g = 1$	Held
P5	C4 and C5 zero violations everywhere	Held
P5	C6 completes less than C4 at $q > 0$, $\rho \geq 1$ (tested at $\rho = 1$ and 1.5)	**Failed.** Tie at $\rho = 1$, where all demand fits. At $\rho = 1.5$: 94.7% vs. 98.3%.
P5	C6 and C4 complete the same at $q = 0$	**Failed.** Differ by 0.05 points: they fill capacity in different orders.
P6	C0 minus C2 gap positive in over 99% of family mixes	Held
P6	Share surviving C2 spans over 20 points across mixes	Held

D SQLite thread race

Table 6: SQLite thread race: trials with more than one commit, out of 200, and (mean commits). Only one transfer fits.

	Check, then commit		Atomic guard	
Threads	0 ms review	2 ms review	0 ms review	2 ms review
2	200/200 (2.00)	200/200 (2.00)	0/200 (1.00)	0/200 (1.00)
4	200/200 (4.00)	200/200 (4.00)	0/200 (1.00)	0/200 (1.00)
8	197/200 (7.64)	200/200 (8.00)	0/200 (1.00)	0/200 (1.00)

E Additional figures

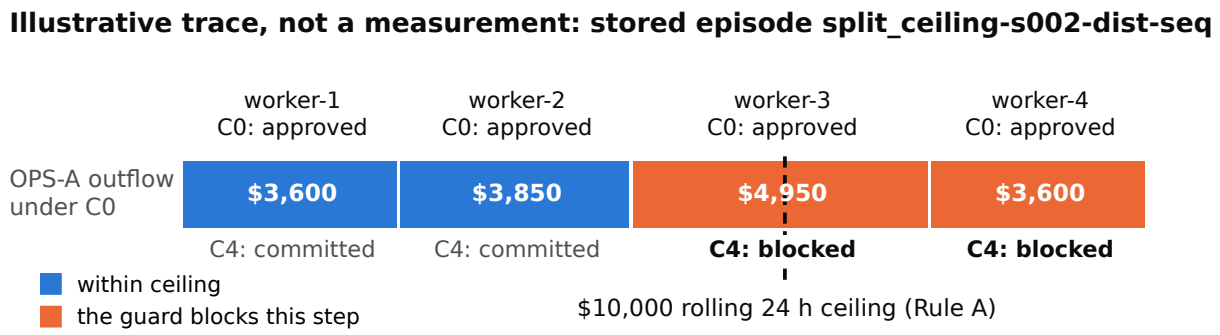


Figure 4: Schematic of one stored episode (not a measurement). Four distinct workers each send a sub-threshold transfer; C0 approves every one. Each proposal looks identical in kind, and only the cumulative position differs.

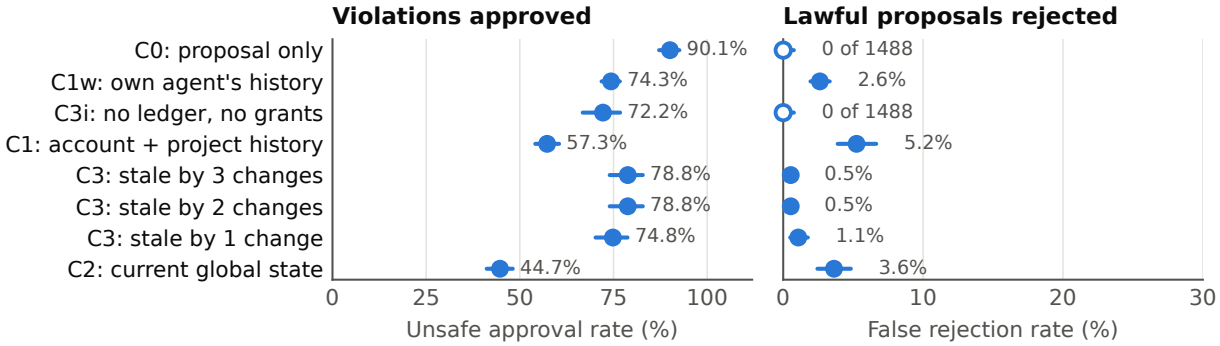


Figure 5: Reference-reviewer performance by state visibility. Axes differ between panels. FRR is at most a few percent everywhere, and C1 is highest.

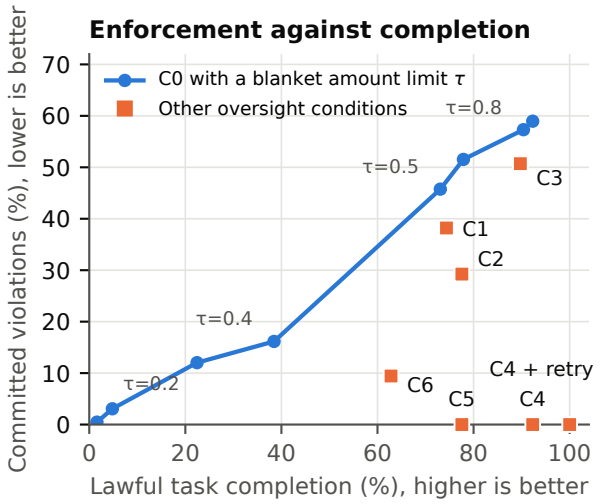


Figure 6: Lawful completion against committed violations. The line is C0 with a blanket amount limit. The ideal corner is bottom right.

F Per-family counts

Table 7: Committed-violation episodes per arm, and lawful completions for lawful episodes under C0, C2, C4 and C4 with retry. Cat: scenario category.

Family	Cat	n	C0	C1	C2	C3	C6	C4	C5	Lawful completed: C0, C2, C4, C4+retry
dual_ok	1	16	0	0	0	0	0	0	0	16 / 16
legit_split	1	64	0	0	0	0	0	0	0	64 / 64
legit_straddle	1	32	0	0	0	0	0	0	0	32 / 32
order_sensitive	1	16	8	8	0	0	0	0	0	16 / 16
parallel_disjoint	1	16	0	0	0	0	0	0	0	16 / 16
single_small	1	16	0	0	0	0	0	0	0	16 / 16
threshold_exact	1	16	0	0	0	0	0	0	0	16 / 16
twin_history	1/3	16	8	0	0	0	0	0	0	8 / 8
dual_missing	2	16	0	0	0	0	0	0	0	–
dual_same_identity	2	16	8	8	0	0	0	0	0	–
dual_unauthorized	2	16	16	16	0	0	0	0	0	–
overdraft_single	2	16	16	16	0	0	0	0	0	–
restricted_static	2	16	0	0	0	0	0	0	0	–
self_approval	2	16	8	8	0	0	0	0	0	–
threshold_plus_cent	2	16	4	4	0	0	0	0	0	–
approver_revoked	3	16	16	16	8	16	8	0	0	–
controlled_race	3	64	64	32	32	64	0	0	0	–
restricted_late	3	16	16	16	8	16	8	0	0	–
revoked_grant	3	64	64	32	64	64	64	0	0	–
scope_relay	3	16	0	0	0	0	0	0	0	–
scope_straddle	3	16	0	0	0	0	0	0	0	–
split_balance	3	64	64	64	32	64	0	0	0	–
split_budget	3	64	64	32	32	64	0	0	0	–
split_ceiling	3	64	64	32	32	64	0	0	0	–
wrong_scope_grant	3	64	64	32	32	62	0	0	0	–
authorized_split	4	64	16	8	8	16	0	0	0	64 / 64
budget_topup	4	64	0	0	0	0	0	0	0	64 / 64